ActorForth – Why Another Language?

- Impedance mismatch between Developer Mental Model of his code and machine execution of that code.
- Imperative style code structures focus too much on the “How” and confusing abstractions hide the “What”.
- Mistaken belief that one cryptoledger/blockchain will “win” just as wrong as one crypto-language will win.
- Make a language to build better languages.

How Did We Get Here?

No skin in the game invites some very serious ethical issues.
We better get our house in order before someone else does first.
(Certifications are NOT the answer!)

Ethereum

- One year after ERC-20 was released there came the DAO.
- On Friday, June 17 2017, programmers experienced real consequences.
- Existential consequences worth >USD$50 million+ >$150 M organization!

How Did We Get Here?

- When you're new Facebook killer app UI crashes at odd times it's ok for your users to just reload their browser.

- When three innocuous lines of code in the wrong order can wipe away millions of dollars of wealth and utterly destroy your organization perhaps you've chosen the wrong architectural model upon which to build your future.

How Did We Get Here?

“When we take the position that it is not only the programmer's responsibility to produce a correct program but also to demonstrate its correctness in a convincing manner, then the above remarks have a profound influence on the programmer's activity: the object he has to produce must be usefully structured.”

— Dijkstra (1970) "Notes On Structured Programming" (EWD249), Section 3 ("On The Reliability of Mechanisms"), p. 6.

Dijkstra’s Revenge!!

Manage to call this recursively
From target contract's function and
You take all the tokens til gas runs out!

Why do air[illegible]es crash & crypto coins destroy

```
function splitDAO(
  uint _proposalID,
  address _newCurat
) noEther onlyToken[illegible]olders returns (bool _success) {

  ...
  // XXXXX Move ether [illegible] how this is done first!
  uint fundsToBeMoved
      (balances[msg.sen[illegible] [illegible]ance) /
      p.splitData[0].tota[illegible]
  if (p.splitData[0].newD[illegible] [illegible]ndsToBeMoved)(msg.sender)
== false) // XXXXX This i[illegible] [illegible]s to run more than once
      throw;

  ...
  // Burn DAO Tokens
  Transfer(msg.sender, 0, ba[illegible]nces[msg.sender]);
  withdrawRewardFor(msg.sen[illegible]r); // be nice, and get his rewards
  // XXXXX Notice the prece[illegible]ing line is critically before the next few
  totalSupply -= balances[msg.sender]; // XXXXX AND THIS IS DONE LAST
  balances[msg.sender] = 0; // XXXXX AND THIS IS DONE LAST TOO
  paidOut[msg.sender] = 0;
  return true;
}
```

Sends tokens to split contract.

Updates local balances
AFTER tokens sent

http://hackingdistributed.com/2016/06/18/analysis-of-the-dao-exploit/

Solidity Contributions to DAO Exploit

- Imperative programming model combined with implicit "magic" object-like model hides non-obvious behaviors
- Lack of atomic paired operations allows contracts to enter indeterminate state

  (Does the term "ledger" not imply something rather fundamental that must be maintained?)
- In this day & age what could possible inspire such an obviously defective model?

  (I'm looking at you javascript & go-lang folk!)

Solidity

- Definition

A Cornell University researcher stated that Solidity was partially to blame for The DAO hack that took place in 2016 He stated: "this was actually not a flaw or exploit in the DAO contract itself: technically the EVM was operating as intended, but Solidity was introducing security flaws into contracts that were not only missed by the community, but missed by the designers of the language themselves."[12][13]

As specified by Wood it is designed around the ECMAScript syntax to make it familiar for existing web developers; unlike ECMAScript it has static typing and variadic return types. Compared to other EVM-targeting languages of the

Wikidata item
Cite this page

Print/export
Create a book
Download as PDF
Printable version

Languages
Català
Deutsch
Русский
Українська
中文
Edit links

Solidity was initially proposed in August 2014 by Gavin Wood;[9][10] the language was later developed by the Ethereum project's Solidity team, led by Christian Reitwiessner. It is one of four languages (the others being Serpent, LLL, Viper (experimental) and Mutan (deprecated)) designed to target the Ethereum Virtual Machine (EVM).[8] For more information on these languages see Ethereum programming languages.

At present, Solidity is the primary language on Ethereum as well as on other private blockchains running on platforms that compete with Ethereum, such as Monax and its Hyperledger Burrow blockchain, which uses Tendermint for consensus. SWIFT has deployed a proof of concept using Solidity running on Burrow.[2][11]

A Cornell University researcher stated that Solidity was partially to blame for The DAO hack that took place in 2016. He stated: "this was actually not a flaw or exploit in the DAO contract itself: technically the EVM was operating as intended, but Solidity was introducing security flaws into contracts that were not only missed by the community, but missed by the designers of the language themselves."[12][13]

Description [edit]

Solidity is a statically-typed programming language designed for developing smart contracts that run on the EVM.[14][15] Solidity is compiled to bytecode that is executable on the EVM. With Solidity, developers are able to write applications that implement self-enforcing business logic embodied in smart contracts, leaving a non-repudiable and authoritative record of transactions.[16][17] Writing smart contracts in smart contract specific languages such as Solidity is referred to as easy (ostensibly for those who already have programming skills).[18]

As specified by Wood it is designed around the ECMAScript syntax to make it familiar for existing web developers; unlike ECMAScript it has static typing and variadic return types. Compared to other EVM-targeting languages of the

Can We Fix It?

"Programs must be written for people to read, and only incidentally for machines to execute."

—Hal Abelson

(Structure and Interpretation of Computer Programs, Second Edition)

"Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

—Brian Kernighan

"The Elements of Programming Style",

2nd edition, chapter 2

Tools & Improvements

- Ethereum Foundation is aware of the issues
- Lots of development going on
- New language Vyper is clearly superior to Solidity (but doesn't go far enough)
- Entire industry of tools to support developers
- The entire VM is being replaced by WebASM (more js influence – yikes!)
- Proof of stake replacing proof of work

Tools & Improvements

- Lots of improvements for "performance"
- Not so many addressing issues raised here
- Can't deploy contracts >~500 lines of code
- OpenZeppelin, et al are band-aids at best and only inject more complexity & size at the end
- Attitude: Build improvements on top of existing architecture

  (can this work and can ETH improve before being overtaken by something new?)

Lessons Learned

“The art of programming is the art of organizing complexity, of mastering multitude and avoiding its bastard chaos as effectively as possible.”

— Dijkstra (1970) "Notes On Structured Programming" (EWD249), Section 3 ("On The Reliability of Mechanisms"), p. 7.

Program testing can be used to show the presence of bugs, but never to show their absence!

— Dijkstra (1970) "Notes On Structured Programming" (EWD249), Section 3 ("On The Reliability of Mechanisms"), corollary at the end.

Alternative Futures

- Provably Correct Algorithms & Code
  - Expressive strong type systems required
  - Cardano is trying to take this approach using Haskell!
  - Will they make it back out of the rabbit hole?
- Contract Complexity 2-3+ orders of magnitude higher
  - Coins are super simple contracts
  - Human contracts far larger and 90% of their logic consists of exception handling
- Declarative & Constraint Based Systems
  - Eliminates most errors caused by imperative programming styles
- Homoiconic languages to build Domain Specific Languages
  - Been around since lisp & forth! Can't we get this right by now?

(Insert your brilliant idea here...)